

Comprehensive Study Guide

CME 295: Transformers & LLMs — Lecture 9 (Final)

Vision Transformers, VLMs, Diffusion LLMs, Open Frontiers

Stanford CME 295 by Afshine & Shervine Amidi · For: Applied AI/ML Engineers

Contents

1	Topic Map	2
2	Deep-Dive Notes	3
2.1	Why Transformers Generalize Across Modalities	3
2.2	Vision Transformer (ViT)	3
2.2.1	ViT End-to-End Pipeline	3
2.3	Vision Language Models (VLMs)	4
2.3.1	Method 1: Decoder-Only Architecture (LLaVA-style)	4
2.3.2	Method 2: Cross-Attention (LLaMA 3 Vision-style)	4
2.3.3	Transformer Applications Across Modalities	5
2.4	Diffusion LLMs: Beyond Autoregressive Generation	5
2.4.1	The ARM Bottleneck	5
2.4.2	Diffusion for Images	5
2.4.3	Masked Diffusion Models (MDM) for Text	6
2.5	Open Research Frontiers	7
2.5.1	Architecture-Level Open Questions	7
2.5.2	The Data Problem	7
2.5.3	Quality-Cost Pareto Frontier	7
2.5.4	Hardware-Native Attention: Analog In-Memory Computing	8
2.5.5	Cross-Modal Pollination	8
2.5.6	Ongoing LLM Challenges	8
2.5.7	Near and Long-Term Trajectory	8
2.5.8	How to Stay Current	9
3	Related Concepts You Should Also Know	10
4	Build Projects	11
5	Explain It Back	12
6	Interview Practice Questions & Answers	13
7	Self-Assessment Test	15

Topic Map

1. Beyond Language: Transformers for Other Modalities

- 1.1 Why Transformers generalize (weak inductive bias)
- 1.2 Applying attention to images: patches as tokens
- 1.3 ViT (Vision Transformer): architecture end-to-end
- 1.4 ViT vs. CNN: inductive biases, data requirements

2. Vision Language Models (VLMs)

- 2.1 Definition: joint image-text understanding and generation
- 2.2 Method 1: decoder-only LLM with patch embeddings (LLaVA-style)
- 2.3 Method 2: cross-attention between image encoder and text decoder (LLaMA 3 Vision)
- 2.4 Multimodal RoPE variants
- 2.5 Transformer families across modalities

3. Diffusion LLMs: Beyond Autoregressive Generation

- 3.1 ARM bottleneck: sequential generation, non-parallelizable inference
- 3.2 Diffusion for images: forward (noise addition) and reverse (denoising)
- 3.3 Masked Diffusion Models (MDM): forward (masking) and reverse (unmasking)
- 3.4 MDM inference: all tokens generated in parallel passes
- 3.5 Advantages: speed ($\sim 10\times$), better suited for some tasks
- 3.6 Challenges: performance gap, adapting ARM techniques

4. Open Research Frontiers

- 4.1 Architecture-level open questions: optimizer choice, normalization, MHA/GQA, activation, MoE
- 4.2 Data: synthetic data collapse, future high-quality data streams
- 4.3 Quality-cost Pareto frontier as the new optimization target
- 4.4 Analog in-memory computing for hardware-native attention
- 4.5 Cross-pollination between modalities (DiT, MM-DiT, MSRoPE)
- 4.6 Ongoing LLM challenges: static weights, hallucination, personalization, interpretability, safety
- 4.7 Near-term and long-term trajectory: agent democratization, OS-level agents

Deep-Dive Notes

Why Transformers Generalize Across Modalities

Inductive Bias

An inductive bias is a set of assumptions built into a model's architecture that cause it to prefer certain types of solutions. For example:

- **CNNs:** Strong inductive biases — translation equivariance (features are detected regardless of location) and locality (nearby pixels interact first). These assumptions are correct for most natural images, making CNNs very data-efficient.
- **Transformers:** Weak inductive biases — attention is computed between all pairs of tokens with no structural assumption about what should interact with what. The model learns these patterns from data.

Why Weak Inductive Bias Is an Advantage at Scale

Weak inductive biases require more data to learn effectively (models don't have built-in shortcuts), but they are also more generalizable — the same Transformer architecture can be applied to text, images, audio, video, and combinations thereof. This “one architecture to rule them all” property is why Transformers dominate across modalities today.

Vision Transformer (ViT)

ViT: Vision Transformer (Dosovitskiy et al., 2020)

Adapts the Transformer encoder to image classification by treating image patches as tokens.

Core idea: An image of size $H \times W$ is divided into $N = HW/P^2$ non-overlapping patches of size $P \times P$ (e.g., $P = 16$). Each patch is flattened and linearly projected to a d -dimensional embedding vector — the image's analog of a word token.

ViT End-to-End Pipeline

ViT Forward Pass for Image Classification

Input: Image of shape (H, W, C) (height, width, channels).

Step 1 — Patch extraction and projection:

Divide into N patches of shape (P, P, C) . Flatten each to a vector of length P^2C . Apply a learned linear projection $\mathbf{W} \in \mathbb{R}^{P^2C \times d}$ to get N patch embeddings $\in \mathbb{R}^d$.

Step 2 — Prepend [CLS] token:

A learned classification token $\mathbf{z}_{\text{CLS}} \in \mathbb{R}^d$ is prepended. Sequence length becomes $N + 1$.

Step 3 — Add position embeddings:

Learned 1D position embeddings (one per patch position) are added element-wise. This encodes the 2D spatial arrangement of patches in 1D sequence order.

Step 4 — Encoder:

Pass through L Transformer encoder blocks (multi-head self-attention + FFN, identical to BERT's encoder).

Step 5 — Classification head:

The output embedding of the [CLS] token is projected through a linear classification head $\rightarrow$ class probabilities.

Table 1: ViT vs. CNN Comparison

Property	ViT	CNN
Inductive bias	Weak (learned)	Strong (locality, equivariance)
Data requirement	High (needs large datasets)	Lower (biases are helpful)
Long-range interactions	Native (all-pairs attention)	Requires many conv layers
Scalability	Excellent (same as LLMs)	Moderate
Architecture universality	Yes (multimodal)	No (domain-specific)

ViT vs. CNN: The Practical Trade-off

For moderate data regimes, CNN-based models (ResNet, EfficientNet) outperform ViT because their inductive biases are correct for natural images. At very large scale (JFT-300M, ImageNet-21K), ViT matches or exceeds CNN performance because the data advantage outweighs the absence of inductive biases. Today, hybrid approaches (CNN for patch extraction + Transformer for global context) are common.

Vision Language Models (VLMs)**VLM: Vision Language Model**

A model that can process both images and text as input and produce text as output. Enables tasks like image captioning, visual question answering (VQA), visual reasoning, and document understanding.

Method 1: Decoder-Only Architecture (LLaVA-style)**LLaVA-Style VLM (Liu et al., 2023)**

Recycle an existing decoder-only LLM by:

1. Encode the image with a pre-trained visual encoder (e.g., CLIP-ViT) into a sequence of visual tokens.
2. Project visual tokens to the LLM's embedding dimension via a learned projection layer (MLP or linear).
3. Concatenate visual tokens + text tokens as a single sequence and feed into the decoder-only LLM.

The LLM attends to both visual and text tokens via its standard self-attention mechanism. No architectural changes required — only the projection layer is new.

Advantage: Minimal architectural changes; can leverage pre-trained LLMs directly.

Limitation: Self-attention over all visual + text tokens is expensive for high-resolution images.

Method 2: Cross-Attention (LLaMA 3 Vision-style)**Cross-Attention VLM (LLaMA 3 Herd, 2024)**

Add dedicated cross-attention layers to the text decoder:

1. Encode image with a visual encoder → visual representations (keys and values).
2. In the text decoder, interleave standard (text) self-attention layers with cross-attention layers.

3. In cross-attention: queries come from the text decoder; keys and values come from the image encoder. Exactly the same as the encoder–decoder cross-attention from Lecture 1.
Advantage: Text and image representations are kept separate, which can improve both quality and efficiency. Image features don't compete for attention heads with text features.
Limitation: More architectural complexity; harder to retrofit onto a pre-trained text-only LLM.

Multimodal RoPE (MSRoPE)

Standard RoPE encodes 1D positions (token index in a sequence). For images, 2D spatial positions must be encoded. Multimodal Scalable RoPE (MSRoPE) extends RoPE to 2D by applying separate rotations for the row and column positions of each image patch. This preserves the relative-distance property (Lecture 2) while handling spatial structure.

Transformer Applications Across Modalities

Table 2: Transformers Across Modalities

Modality	Architecture Variant	Example Models
Text generation	Decoder-only	GPT-4, LLaMA 3, Mistral
Vision understanding	ViT (encoder-only)	ViT-L, CLIP vision encoder
Image generation	DiT, MM-DiT	Stable Diffusion 3, FLUX
Vision + language	VLM (decoder-only or cross-attn)	LLaVA, LLaMA 3 Vision, GPT-4V
Speech	Transformer encoder/decoder	Whisper, Wav2Vec 2.0
Recommendation	Transformer-based	Various industry systems

Diffusion LLMs: Beyond Autoregressive Generation

The ARM Bottleneck

Autoregressive Modeling (ARM) Bottleneck

Current LLMs generate text token by token, left to right. To generate position t , the model must first complete positions $1, \dots, t-1$. This is inherently sequential — no parallelism at inference time, despite training being parallelizable (teacher forcing).

Consequence: Generating n tokens requires n separate forward passes through the full model. This is the primary reason LLM inference is slow and expensive for long outputs.

Diffusion for Images

Image Diffusion: Forward and Reverse Processes

Forward process: Gradually add Gaussian noise to an image over T timesteps until the image becomes pure noise ($\mathcal{N}(0, I)$). Analytically tractable.

Reverse process: Train a neural network (denoiser) to predict and remove noise at each timestep, iteratively reconstructing the original image from pure noise.

Key insight: Noise is easy to sample from any distribution. The model learns a transformation from the tractable noise distribution to the complex data distribution.

Michelangelo analogy: The sculpture (clean image) is already within the marble block (noise). Diffusion chisels away the superfluous material (noise) step by step.

Masked Diffusion Models (MDM) for Text

MDM: From Image Noise to Token Masks

Discrete tokens cannot be corrupted with continuous Gaussian noise. The adaptation:

Forward process (masking): Replace each token independently with a [MASK] token with some probability. At the maximum corruption level, all tokens are masked.

Reverse process (unmasking): Train a model to predict the original tokens at masked positions. Unlike ARM, the model attends to *all* positions (including unmasked tokens) bidirectionally.

ARM inference: T forward passes for T tokens vs. MDM inference: $\ll T$ forward passes

MDM Inference in Practice

At inference time, start with all tokens masked. In each forward pass:

1. The model predicts probabilities for all masked positions simultaneously.
2. Unmask the most confident predictions (or a fixed fraction per step).
3. Repeat with the remaining masked tokens, now conditioned on the unmasked ones.

After $k \ll T$ passes, all tokens are filled in. This is sometimes called “masked sampling” or “iterative decoding.”

Table 3: ARM vs. MDM Comparison

Property	ARM (current LLMs)	MDM (Diffusion LLMs)
Training objective	Next-token prediction	Predict masked tokens
Inference parallelism	Sequential (token by token)	Parallel (all positions simultaneously)
Inference speed	$1\times$ (baseline)	$\sim 10\times$ faster
Attention during inference	Causal (past only)	Bidirectional (all tokens)
Task suitability	Strong for all sequential tasks	Better for certain structured tasks
Maturity	Very mature	Emerging (2024–2025)
Performance	State of the art	Currently behind ARM

Why MDM Might Be Better for Some Tasks

Tasks requiring global coherence — where the beginning and end of an output must be consistent — are naturally suited to bidirectional generation. Legal document generation, code with paired open/close structures, and structured data outputs (JSON, tables) may benefit from seeing all positions simultaneously.

MDM Current Challenges

- **Performance gap:** On most standard benchmarks, MDMs are still behind ARM-based models of similar scale.
- **Adapting ARM techniques:** The entire ecosystem of ARM optimizations (KV caching, speculative decoding, GRPO/PPO, SFT on sequential reasoning chains) must be rethought for bidirectional masked diffusion.

- **Inference strategy:** How many passes, how many tokens to unmask per pass, and how to handle sampling diversity are open engineering questions.

Open Research Frontiers

Architecture-Level Open Questions

What Is Still Being Debated at the Model Level

Despite thousands of LLM papers, fundamental design choices are still actively studied:

- **Optimizer:** AdamW is standard, but MuonClip (a second-order method with improved stability) and others are showing promise (e.g., Kimi K2).
- **Normalization:** Pre-LN vs. Post-LN vs. RMSNorm vs. no norm — still varies across architectures.
- **Attention:** MHA vs. MQA vs. GQA — each lab makes different choices based on their memory/quality trade-off.
- **Activation functions:** SiLU (used in LLaMA) vs. GeLU vs. ReLU — small differences, studied carefully.
- **MoE vs. dense:** Many labs are converging toward MoE (DeepSeek, Mistral) but pure dense models (LLaMA) remain competitive.
- **Number of layers:** Depth vs. width trade-off is still empirically studied.

The key point: there is no universally optimal configuration. The best choice depends on hardware, data, and target task distribution.

The Data Problem

Model Collapse from Synthetic Data (Shumailov et al., 2023)

As LLMs are used to generate training data for the next generation of models, there is a risk of **model collapse**: the output distribution of the generator is slightly different from the original data distribution; when trained on this shifted distribution, the next model drifts further; iterating this process compresses the diversity of the data distribution, causing the model to “forget” rare but important patterns.

Practical implication: High-quality human-generated data remains irreplaceable as an anchor. Synthetic data must be carefully filtered and mixed with human data. The ceiling on purely synthetic pre-training is an open research question.

Quality-Cost Pareto Frontier

The New Optimization Target: Pareto Frontier

Early LLM development focused on maximizing performance (MMLU score, reasoning benchmark). The shift now is toward the **quality-cost Pareto frontier**: finding the cheapest model that achieves a given quality level, or equivalently, the highest-quality model at a given cost.

This is why quantization, MoE, distillation (Lecture 6), QLoRA (Lecture 4), and GQA (Lecture 2) are so important in practice — they move models closer to the Pareto frontier without sacrificing quality.

Hardware-Native Attention: Analog In-Memory Computing

Analog In-Memory Computing for Attention (Leroux et al., 2025)

Current GPUs are optimized for matrix-matrix multiplications (tensor cores) and have limited support for the KV cache memory access patterns that attention requires.

Idea: Design hardware where:

- The KV cache is stored in dedicated analog memory cells.
- Attention dot products are computed using analog signals (current or voltage) rather than digital arithmetic.

Results (reported): Up to $\sim 100\times$ latency improvement and $\sim 70,000\times$ energy savings compared to Nvidia H100 for attention-specific operations.

Caveat: Analog computing has lower precision than digital. Integration with the full LLM inference stack remains a research challenge.

Cross-Modal Pollination

The field has stopped treating modalities as separate problems. Architectural ideas now flow freely between them:

- Diffusion training objective $\xrightarrow{\text{adapted to text}}$ Diffusion LLMs (MDMs)
- Transformer architecture $\xrightarrow{\text{adapted to images}}$ DiT (Diffusion Transformer, Peebles et al., 2022)
- Transformer + Diffusion $\xrightarrow{\text{multimodal}}$ MM-DiT (Stable Diffusion 3)
- RoPE $\xrightarrow{\text{adapted to 2D images}}$ MSRoPE (Multimodal Scalable RoPE)
- ViT embeddings $\xrightarrow{\text{input to LLM}}$ VLMs (LLaVA, LLaMA 3 Vision)

Ongoing LLM Challenges

Five Unsolved Problems (as of Dec 2025)

1. **Static weights:** Model knowledge is frozen after training. No continuous learning from new interactions without retraining. Current workarounds: RAG, tool calling, periodic fine-tuning.
2. **Hallucination:** Models confidently state incorrect facts. Mitigation: RAG, factuality training, structured output, retrieval verification. No complete solution.
3. **Personalization:** Adapting to an individual user's style, preferences, and context without compromising privacy. In-context history helps but is limited by context length.
4. **Interpretability:** We cannot reliably explain why a model produces a specific output. Mechanistic interpretability (a research direction) makes progress but remains far from practical utility.
5. **Safety:** Preventing misuse (Lecture 7), ensuring alignment (Lectures 5–6), and avoiding catastrophic failure modes (especially for autonomous agents). Active area with significant urgency.

Near and Long-Term Trajectory

- **Tomorrow:** Agent democratization — tools like Google Workspace Studio bring agentic AI to non-technical users at scale.

- **Near term:** Browser-level agents (OpenAI Atlas, Nov 2025) that can perceive and act on the web. OS-level integration is the next frontier.
- **Long term:** Truly autonomous agents with broad task delegation. Customer service that actually resolves issues (rare today).

How to Stay Current

Resources for Staying Up-to-Date

Papers:

- arXiv cs.CL (Computation and Language) — daily releases.
- Venues: NeurIPS, ICML, ICLR (general ML); ACL, EMNLP (NLP-specific).

Code:

- Authors' GitHub repositories (linked in papers).
- Hugging Face “trending” — real-time popularity signal.

Miscellaneous:

- Twitter/X: academic researchers and industry leads.
- YouTube: Two Minute Papers, Yannic Kilcher (theory), Andrej Karpathy, HuggingFace, Google Developers (practical).
- Company blogs: Anthropic, OpenAI, Google DeepMind, Meta AI, Microsoft AI, Apple ML.
- Stanford NLP group blog.

Reference: Stanford CME 295 VIP cheatsheet on GitHub (github.com/afshinea/stanford-cme-295-transformers-large-language-models).

Related Concepts You Should Also Know

1. **CLIP (Contrastive Language-Image Pre-Training, Radford et al., 2021)**: Joint pre-training of a vision encoder and text encoder on 400M (image, text) pairs. The objective: bring matching pairs close and push non-matching pairs apart in embedding space (contrastive loss). CLIP’s visual encoder is the most common image encoder used in VLMs (LLaVA, etc.). Enables zero-shot image classification.
2. **DiT (Diffusion Transformer, Peebles & Xie, 2022)**: Replaces the U-Net backbone in image diffusion models with a Transformer. Scales much better than U-Net, enabling the current generation of high-quality image generators (Stable Diffusion 3, FLUX).
3. **DINO / DINOv2 (Meta, 2021/2023)**: Self-supervised vision Transformers trained without labels via self-distillation. The resulting features are highly transferable across tasks. Often used as visual encoders in VLMs.
4. **Model Collapse (Shumailov et al., 2023)**: The theoretical and empirical observation that repeatedly training on model-generated data causes progressive quality degradation. The distribution of generated text is slightly different from human text; compounding this difference over training iterations erases tail-distribution diversity.
5. **MuonClip**: A second-order optimizer with clipping to prevent gradient explosions. Combines Nesterov momentum with a gradient orthogonalization step. Reported to improve training stability for very large models. Used by Kimi K2 (2025).
6. **Grokking**: A phenomenon where a model first memorizes training data (achieving near-perfect train accuracy but near-random test accuracy), then suddenly generalizes with further training. Observed in Transformer-based models on algorithmic tasks. Theoretically important for understanding emergent behaviors.
7. **Selective State Space Models (Mamba)**: An alternative to Transformers for sequence modeling, using selective state spaces to achieve $O(n)$ inference complexity (vs. $O(n^2)$ for attention). Emerging as a competitor for certain sequence modeling tasks, though has not yet matched Transformer performance at large scale.
8. **Mechanistic Interpretability**: A research direction (Anthropic, EleutherAI, others) that tries to reverse-engineer what specific computations individual neurons and circuits perform inside a Transformer. Tools include activation patching, probing classifiers, and sparse autoencoders.
9. **Test-Time Training (TTT)**: Updating model weights at inference time using gradient descent on a prediction task derived from the test input. Bridges the gap between static weights and continuous learning. Active research area (TTT layers, etc.).
10. **Tokenizer-Free Models (ByT5, etc.)**: Operate at the byte level (Lecture 2’s ByT5). Eliminates tokenization and its associated issues (OOV, byte-order sensitivity). Promising for multilingual and code tasks but produce very long sequences.

Build Projects

1. **ViT from Scratch** (*5–7 hours, strong foundational signal*)

Implement a ViT-Tiny (6 layers, 3 heads, patch size 16) in pure PyTorch from scratch. Train on CIFAR-10 (smaller than ImageNet, feasible on a single GPU). Implement: patch extraction, linear projection, [CLS] prepend, positional embedding addition, encoder stack, classification head. Compare with a ResNet-18 baseline trained on the same data under the same budget. Plot loss curves and top-1 accuracy. Key finding: ViT needs much more data than ResNet at this scale. This directly demonstrates the weak-inductive-bias trade-off discussed in class.

2. **Simple VLM with LLaVA Architecture** (*4–6 hours, high signal*)

Build a minimal VLM using a pre-trained CLIP-ViT encoder and a small open-source LLM (e.g., LLaMA-3.2-1B). The connector: a two-layer MLP projecting CLIP’s 768-dim visual tokens to LLaMA’s 2048-dim space. Fine-tune only the connector on a small VQA dataset (e.g., VQAv2 subset). Evaluate on visual question answering. Demonstrates Method 1 from class with real components.

3. **MDM Exploration on a Toy Sequence Task** (*4–5 hours, frontier signal*)

Using the `mdlm` or `mlm-scoring` library (or implementing from scratch on a toy task), compare ARM vs. MDM on a structured sequence generation task (e.g., generating valid JSON, or arithmetic expressions). Measure: (a) inference speed (tokens/second), (b) structural validity rate (does the output parse?), (c) semantic correctness. This is a direct test of the “better suited for some tasks” claim from class.

Explain It Back

1. Explain to a computer vision engineer who has never used Transformers why ViT works, what [CLS] does, and why ViT needs more data than ResNet to achieve comparable performance. Use the inductive bias argument precisely.
2. Compare LLaVA-style (Method 1) and cross-attention-style (Method 2) VLMs. For each: what gets concatenated/attended over what, where is the image information injected, and what are the practical trade-offs for a team building a production VLM?
3. Explain masked diffusion models to someone who knows ARM-based LLMs well. Cover: what the forward and reverse processes are, why masking is used instead of Gaussian noise, how inference works at a high level, and what the speed advantage is.
4. You are giving a 5-minute talk to a startup founder about what will change in AI in the next 2–5 years. Using the trajectory from this lecture, cover: what’s becoming commoditized, where the frontier is, and what the three biggest unsolved problems are.

Interview Practice Questions & Answers

L1 — How does ViT convert an image into tokens for the Transformer?

Answer: ViT divides the input image of size $H \times W$ into $N = HW/P^2$ non-overlapping patches of size $P \times P$ (typically $P = 16$). Each patch is flattened into a vector of length P^2C (where C is the number of channels) and linearly projected to a d -dimensional embedding. A learnable [CLS] token is prepended, giving $N + 1$ tokens. Learned 1D positional embeddings are added to encode spatial position. The resulting sequence is fed into a standard Transformer encoder. The [CLS] token's output is used for classification via a linear head — exactly as in BERT for text classification.

L1 — What is the fundamental limitation of autoregressive generation that diffusion LLMs address?

Answer: Autoregressive models generate tokens sequentially — each token requires a full forward pass conditioned on all previously generated tokens. Generating n tokens requires exactly n forward passes, none of which can be parallelized (unlike training, which uses teacher forcing to parallelize over all positions). This makes inference slow and expensive for long outputs. Masked Diffusion Models address this by generating all token positions simultaneously in each forward pass: they start with fully masked outputs and iteratively unmask positions in $k \ll n$ passes. The speed advantage is approximately $n/k \approx 10\times$ in practice, at the cost of some quality loss relative to state-of-the-art ARM models.

L2 — Compare the two main approaches to building VLMs. Which would you choose for a new project?

Answer: Method 1 (LLaVA-style): image patches are encoded by a visual encoder (e.g., CLIP-ViT), projected to the LLM's embedding dimension via an MLP, and concatenated with text tokens as input to a standard decoder-only LLM. No architectural changes to the LLM itself. Simpler to implement, easier to retrofit onto a pre-trained LLM. However, long image sequences consume a lot of context. Method 2 (cross-attention, LLaMA 3 Vision-style): a separate visual encoder feeds its representations as keys and values into dedicated cross-attention layers in the text decoder. Text and image information stay separate until cross-attention. More expressive and efficient for high-resolution images, but architecturally more complex. Choice: for a quick prototype or if starting from a pre-trained LLM with no code changes, Method 1. For production at scale with long-context images, Method 2.

L2 — What is model collapse and why does it matter for synthetic data?

Answer: Model collapse (Shumailov et al., 2023) is the progressive degradation of a model trained on data generated by another model. Each generation of model produces output with a slightly different distribution from the original human data. Training the next model on this output compresses the distribution further, discarding rare but important patterns. Iterating this process causes the model to converge to a narrow, over-simplified distribution — it “forgets” the full diversity of human language. This matters because: (1) LLM-generated synthetic data is increasingly used for SFT and preference tuning; (2) as the internet fills with AI-generated text, pre-training data quality degrades; (3) purely synthetic data pipelines must be carefully anchored with human-generated data to prevent collapse.

L1 — Why do CNNs need less data than ViTs, and when does this advantage disappear?

Answer: CNNs have strong inductive biases: convolutional filters are translation-equivariant (a feature detector works regardless of where it appears in the image) and focus on local relationships (early layers detect edges; later layers combine them). These biases are correct for most natural images, giving CNNs a strong “head start” with limited data. ViTs have weak inductive biases — they learn from scratch which positions should interact and what features matter. This requires more data to discover patterns that CNNs get for free from architecture design. The advantage disappears at very large scale: with hundreds of millions of training examples, ViTs learn the same invariances from data and match or exceed CNN performance. Google’s original ViT paper showed ViT-L exceeding CNN baselines only when pre-trained on JFT-300M (300 million images), not on ImageNet alone (1.3 million).

L3 — Explain what RoPE is (from Lecture 2) and how it is adapted for 2D image positions in VLMs.

Answer: RoPE (Rotary Position Embeddings) encodes position by rotating the query and key vectors in the attention layer by an angle proportional to the token’s position. The key property: $\tilde{q}_m^\top \tilde{k}_n = f(m - n)$ — the dot product is a function of relative distance only, because $R_m^\top R_n = R_{n-m}$ (rotation matrices compose by subtraction). For text (1D), a single angle per dimension pair suffices. For images in VLMs, tokens have 2D positions (row and column). MSRoPE (Multimodal Scalable RoPE) extends this by applying separate rotation angles for the row and column dimensions of each image patch’s position, independently for each feature dimension pair. This allows the attention mechanism to capture 2D spatial relationships between image patches while preserving the relative-distance property.

Self-Assessment Test

Scoring: 16–20 = solid mastery; 12–15 = revisit weak spots; <12 = re-study.

Multiple Choice (1 point each)

Q1. In ViT, image patches are analogous to:

- (a) Convolutional filters (b) Word tokens in a text Transformer (c) Attention heads (d) Layer normalization operations

Q2. The main limitation of CNNs compared to ViTs is:

- (a) CNNs cannot process color images (b) CNNs have strong inductive biases that limit generalizability across modalities at scale (c) CNNs require more data than ViTs (d) CNNs cannot be combined with Transformers

Q3. In the LLaVA-style VLM, image information is injected by:

- (a) Adding image embeddings to the LLM’s positional encoding (b) Projecting visual tokens to the LLM embedding dimension and concatenating with text tokens (c) Using cross-attention between image encoder outputs and text decoder queries (d) Fine-tuning the visual encoder on the target task

Q4. MDM forward process (for text) involves:

- (a) Adding Gaussian noise to token embeddings (b) Masking tokens with some probability (c) Predicting the next token autoregressively (d) Quantizing embeddings to lower precision

Q5. The approximate inference speed advantage of MDMs over ARM is:

- (a) $2\times$ (b) $5\times$ (c) $10\times$ (d) $100\times$

Q6. Model collapse when training on synthetic data refers to:

- (a) The model’s weights collapsing to zero (b) Progressive loss of distribution diversity when models train on their own outputs iteratively (c) Training instability from high learning rates (d) The reward model collapsing during RLHF

Q7. Analog in-memory computing for attention aims to address:

- (a) The $O(n^2)$ computation cost of attention (b) The energy and latency cost of KV cache memory access patterns, which dominate GPU inference (c) Hallucination in LLM outputs (d) The training instability of large models

Q8. MSRoPE (Multimodal Scalable RoPE) extends standard RoPE by:

- (a) Using larger embedding dimensions (b) Applying separate rotations for row and column positions of image patches (c) Removing the position encoding entirely for images (d) Using learned rather than fixed rotation angles

Short Answer (1.5 points each)

Q9. List the five steps of a ViT forward pass for image classification, in order.

Q10. What is the key difference between Method 1 (LLaVA-style) and Method 2 (cross-attention) VLMs in terms of how image and text representations interact?

Q11. Explain the Michelangelo analogy for diffusion models. What does the marble represent and what does chiseling represent?

Q12. List four architecture-level design choices that are still actively debated in LLM development (as of Dec 2025).

Q13. Name and briefly describe two of the five ongoing LLM challenges listed in the lecture.

Q14. Why does ViT require more training data than a CNN of comparable size? Be specific about the role of inductive biases.

Scenario (2 points)

Q15. Your company wants to build a multi-lingual, multimodal customer support system that handles text queries, screenshots of error messages, and product images. It must run at <100ms latency at scale. Outline your architecture choices covering: (a) which VLM approach (Method 1 or 2) and why; (b) whether you would use ARM or MDM for text generation and why; (c) how you would handle the multi-lingual requirement (tokenizer + embedding choices); (d) one inference optimization from earlier in the course that applies here; (e) one evaluation strategy from Lecture 8.

Answer Key

Q1. (b) Patches are the image analog of text tokens — the atomic units fed into the Transformer.

Q2. (b) CNNs’ strong inductive biases (locality, equivariance) limit their ability to learn arbitrary interaction patterns across positions and modalities. At large scale, ViTs’ data-learned patterns match CNN biases while being more general.

Q3. (b) Visual tokens are encoded by a visual encoder, projected via MLP to the LLM’s embedding dimension, and concatenated with text tokens as input to the decoder.

Q4. (b) MDM forward process: mask tokens (replace with [MASK]) with some probability. Unlike image diffusion, which uses continuous Gaussian noise, text tokens are discrete so masking is the natural analog.

Q5. (c) Approximately $10\times$ faster, as reported in the lecture’s discussion.

Q6. (b) When models train on outputs of other models, each generation’s output distribution differs slightly from true human data. Iterating compounds this drift, causing progressive loss of diversity.

Q7. (b) Standard GPUs are optimized for matrix-matrix operations (tensor cores) but the KV cache requires frequent reads/writes that dominate inference cost. Analog in-memory computing stores the KV cache in dedicated cells and computes attention via analog signals, potentially reducing this bottleneck by orders of magnitude.

Q8. (b) Standard RoPE encodes 1D token positions. For 2D image patches, MSRoPE applies separate rotation angles for the row and column positions, preserving the relative-distance property in 2D.

Q9. (1) Divide image into patches, flatten, and linearly project to d -dimensional embeddings. (2) Prepend a learnable [CLS] token. (3) Add learned positional embeddings to all tokens. (4) Pass through L Transformer encoder blocks. (5) Project the [CLS] output through a linear classification head to class probabilities.

Q10. Method 1 (LLaVA): visual tokens and text tokens are concatenated into a single sequence; the LLM’s self-attention operates over all of them together — image and text representations mix inside the same attention computation. Method 2 (cross-attention): visual tokens stay in the image encoder; text and image representations interact only through dedicated cross-attention layers where text queries attend to image keys and values — the two modalities are kept separate except at these specific interaction points.

Q11. The marble = the noise distribution (easy to sample from, fully random). The sculpture = the target data (a specific image or text sequence). Chiseling = the learned denoising / unmasking operation that progressively reveals the target from noise. The idea: the final output is “already present” within the noise in some sense; diffusion learns to extract it step by step.

Q12. Any four of: optimizer choice (AdamW vs. MuonClip vs. others), normalization strategy (Pre-LN vs. RMSNorm vs. none), attention architecture (MHA vs. MQA vs. GQA), activation functions (SiLU vs. GeLU vs. ReLU), MoE vs. dense, number of layers.

Q13. Any two of: (1) *Static weights* — model knowledge freezes at training cutoff; no continuous learning from new interactions. (2) *Hallucination* — model confidently states incorrect facts; no reliable mitigation. (3) *Personalization* — adapting to individual users without sacrificing privacy. (4) *Interpretability* — cannot reliably explain model outputs. (5) *Safety* — preventing misuse and ensuring alignment of increasingly autonomous agents.

Q14. CNNs have strong inductive biases: convolutional filters are translation-equivariant and

focus on local patterns. These biases are correct for natural images, giving CNNs useful shortcuts that require minimal data to activate. ViTs have no such biases — all-pairs attention must learn from scratch which positions are informative and which patterns matter. This learning requires far more examples. Concretely: ViT-Large pre-trained on ImageNet-1k (1.3M images) underperforms ResNet-50; pre-trained on JFT-300M (300M images), it exceeds ResNet-50.

Q15. (a) **VLM: Method 1 (LLaVA-style)** for simplicity and speed. Cross-attention (Method 2) is architecturally more complex and slower to build. With $<100\text{ms}$ latency constraint, you want minimal new components. Use a pre-trained CLIP-ViT encoder + an MLP connector into a multilingual LLM. (b) **ARM over MDM** for text generation. MDMs are $\sim 10\times$ faster but currently significantly behind in quality and the ecosystem is immature. For a customer support system, answer quality directly affects user satisfaction; the MDM speed advantage may not compensate for quality loss. Revisit MDMs in 12–18 months. (c) **Multilingual:** use a multilingual tokenizer (SentencePiece trained on 100+ languages, as in mT5/LLaMA 3 with 128K vocab). Ensure the visual encoder (CLIP) was trained with multilingual captions; otherwise image-text alignment may be weaker for non-English text in images. (d) **Inference optimization:** QLoRA or INT8 quantization to reduce model size and latency. GQA (grouped-query attention) to shrink KV cache, enabling higher throughput per GPU. (e) **Evaluation:** LLM-as-a-Judge with three dimensions — accuracy (did it answer the query correctly?), multimodal grounding (did it use the image information?), safety (did it avoid harmful or misleading responses?). Use binary pass/fail, rationale-before-score, and calibrate weekly against 50 human-reviewed samples.

End of Study Guide — Lecture 9 (Final Lecture of CME 295)

Source: Stanford CME 295, Fall 2025. Afshine & Shervine Amidi.

Congratulations on completing the full course. Good luck on the final!